Volume 02 - The /api/predict Request, Line Allocation, and JSON Contract

Source revision: 4405d167fc323f33be76e2c7f6614fd21cc574a8 **Primary source:** app/routes.py:234-420; connected caller app/static/js/dashboard.js:154-201; connected callee app/scheduling_optimizer.py:252-306. Complete source excerpts for the whole API route and both small endpoints are inserted in the PDF appendix from this revision.

1. The request contract

The endpoint is registered by @app.route('/api/predict', methods=['GET', 'POST']). The primary dashboard sends a POST with Content-Type: application/json and a JSON body shaped like:

{"station":"Rajiv Chowk","date":"2026-10-14","time":"09:00","line":""}

The endpoint also supports GET query parameters. It returns a JSON object with status, station metadata, date/time and calendar context, selected-time and hourly demand, the full profile, scheduling headline/all-line outputs, travel estimates, and provenance strings. Date/time parse errors return HTTP 400 with {status: "error", message: ...}. Other unexpected errors are not caught at the endpoint and may become HTTP 500 responses.

2. Start of api_predict() - select input and parse time (source lines 234-253)

1. The decorator binds both GET and POST to this function. The function docstring describes dynamic frontend use but does not restrict method. 2. request.get_json(silent=True) asks Flask to parse JSON; silent=True suppresses parse errors by returning None. or request.args selects query parameters if JSON is absent or false-like. Thus an empty JSON object also falls back to query args. 3. data.get(...) uses defaults for station, date, time, and line. station is assumed to be a string because .strip() and .rstrip() are called immediately. The code removes trailing footnote symbols then whitespace. date_str, time_str, line_filter are not type-checked. 4. _parse_datetime produces local scenario date/hour/minute. Only ValueError from this block is caught and converted into JSON plus 400. Attribute/type errors caused by malformed JSON shapes or non-string values are not caught here. 5. Calendar context is computed from the parsed date. service_open is checked with both hour and minute, then stored into that mutable context dictionary. Station metadata is looked up even for a closed time.

Verified risk: this route has no allowlist check for station names before scheduling. get_station_info() returns a fallback with empty all_lines for unknown station names; optimize_station_schedule() then builds no line recommendations and calls max() on an empty list. That raises ValueError and is not caught by /api/predict. Validate station against ALL_STATIONS before forecasting/scheduling to return a useful 400 instead.

3. Forecast section (source lines 254-258)

Three distinct quantities are prepared:

• time_footfall: minute-interpolated selected-time estimate, only if the service gate is open; else literal integer zero.

• hourly_footfall: model estimate for the selected whole hour, also zero when closed.

• profile_24h: the 19-point daily profile from 05:00 through 23:00, calculated regardless of selected-time closure.

The variable name profile_24h is historical shorthand. It is not 24 observations: the implementation has 19 hourly labels. Therefore a selected time of 02:02 should have zero selected-time demand and closed service while the same response can still carry a daytime chart. The chart is not a prediction that trains run at 02:02.

The forecasting helper includes its own service check in minute interpolation, but the route performs a service check before calling it. This is redundant defensive gating for selected-time count; direct hourly prediction itself only returns zero outside hours 05-23, not at 23:30 boundary by minute. Route-level gate is what handles 23:31 as closed.

4. Station/line scheduling (source lines 259-270)

The API passes:

• station and primary line from cleaned station lookup;

• every line listed for that station;

• selected hour and Python weekday number (Monday=0 through Sunday=6);

• total_footfall=hourly_footfall (the whole-hour station demand, not the minute-interpolated figure);

• workday and service-open flags.

optimize_station_schedule() splits total demand over line shares at interchanges, calculates per-line recommendations, and returns an object containing line_recommendations, primary_recommendation (the highest-demand line), is_interchange, and total station demand. It performs closed-service overrides inside the optimizer too. The route then handles headline display/aggregation.

5. Choose the headline: selected line vs station overview (source lines 271-324)

Initially filtered_rec points to the optimizer's primary recommendation. station_summary is true only if there is no line filter and the schedule says this is an interchange.

If line_filter is truthy, the route loops over line recommendations, comparing lowercase line names. A match replaces filtered_rec; if there is no match, the previous primary line remains silently. **Potential UX risk:** invalid line names are not reported, so the response can look like a line-specific selection while actually showing the primary line. Validate membership and return 400 or include a warning.

If it is an unfiltered interchange, the API constructs an aggregate headline:

• total_demand uses optimizer station total.

• recommended_capacity and existing_capacity sum per-line estimated clearance capacity. This combines corridor capacities for the station overview.

• utilization = total_demand / recommended_capacity * 100, with zero if capacity is false-like.

• Badge thresholds are strict > comparisons: above 105 critical; above 88 high; above 65 moderate; otherwise low. So exactly 65 is labeled low; exactly 88 is moderate; exactly 105 is high.

• Trains/hour add line frequencies together. Combined headway is deliberately None, because parallel/interchange lines do not have one meaningful shared headway.

• Average wait is a demand-weighted average: sum(wait_i * demand_i) / max(total_demand, 1). The denominator floor prevents division by zero. If total demand is zero but line waiting values were nonzero, this floor would not make it a mathematically weighted average; closed handling follows later.

• fleet_size_available sums line fleet sizes. The displayed deployed count sums each line recommendation's deployed fleet estimate.

• The narrative uses comma-grouped values and labels the capacity estimate; it explicitly says the value is not a live passenger count.

Illustrative calculation only: suppose the lines' hourly demands were 4,000 and 3,000 and their recommended clearance capacities were 12,000 and 8,000. Combined utilization is (7,000 / 20,000) * 100 = 35%. This is not an actual request result; it only demonstrates the formula.

6. Closed-service override (source lines 326-345)

After all other headline decisions, if the selected minute is closed, the route makes a shallow dictionary copy ({**filtered_rec, ...}) and overwrites service-dependent values. This preserves other keys such as is_peak and any fields not explicitly replaced, while setting line demand, trains/hour and capacity to zero; headways and wait to None; utilization to None; risk to SERVICE_CLOSED; badge to Metro closed; and a closure explanation.

This is why the frontend can show N/A utilization instead of zero percent: no operating capacity denominator exists when no trains are running. It also ensures that station summary and line-focus headline both show closed state. schedule_res['line_recommendations'] remains available to the table and is independently closed-overridden inside optimize_station_schedule().

7. Travel graph integration (source lines 347-357)

The endpoint asks shortest_times_from(travel_graph, station) for a dictionary of destination station -> estimated minutes. It maps each destination to a line using dict(zip(stations_df['Station'], stations_df['Line'])); if no line is mapped, it emits Metro Network. It sorts by minutes and builds objects {destination, travel_time_min, line} with time rounded to one decimal place.

Unlike the legacy HTML route, this graph call is not wrapped in a try/except. An unexpected graph exception will abort the request. The output is a shortest-time estimate, not path steps or a timetable journey, and it does not depend on selected clock time/service-open flag.

8. Calendar enrichment and response body (source lines 359-398)

The API adds hour_band, selected whole-hour demand, and daily peak hour to the calendar dictionary, then returns jsonify(...):

• status is "success" on this path.

• station and station_info expose canonical request key and metadata.

• datetime serializes date/day/time/hour/minute plus optimizer peak flag and service-open boolean.

• calendar_context includes weekday/workday/occasion/source plus UI helper fields.

• footfall distinguishes selected-minute interpolation, selected-hour demand, daily total, and daily peak.

• scheduling exposes headline, line-level schedules and interchange boolean.

• profile_24h carries chart series/labels/line curves.

• travel_times is sorted destination data.

• data_provenance explicitly marks topology, capacities, headways, footfall and line share as project estimates or assumptions.

Why types matter: a client may treat capacity_utilization_pct as a number when open and null when closed; headways are numbers when applicable and null for all-line/closed headlines; the frontend must branch instead of blindly appending % or m.

9. api_stations() and api_model_metrics() (lines 401-420)

api_stations() repeats much of the metadata serialization used by index(): iterate rows, split All_Lines, convert nullable coordinates to float-or-null, and return {stations: [...]}. It is a compact discovery endpoint; the current dashboard also receives stations_meta embedded in the template, so this endpoint is not the only way the browser obtains station metadata.

api_model_metrics() simply returns get_model_evaluation_summary() as JSON. It exposes saved metrics and provenance labels; it does not run training or compute new scores on a request.

10. End-to-end request trace with illustrative values

Input: Rajiv Chowk, 2026-10-14, 09:15, no selected line.

1. Browser serializes the controls into JSON. Request ID increments before fetch. 2. Flask parses date/time; dt_obj is a naive 09:15 local scenario, and weekday is Wednesday (2). 3. Calendar helper supplies workday/occasion context; service helper returns true for 09:15. 4. Inference produces model estimates at 09:00 and 10:00; minute interpolation is p09 + (p10-p09)*(15/60) then rounded and clamped nonnegative. The example gives the formula, not repository-verified passenger values. 5. The line schedule call receives selected-hour demand, not the 09:15 interpolated number. 6. Since Rajiv Chowk is an interchange and no line is selected, line recommendations are aggregated. Each line's estimated clearance capacity is summed; utilization is station demand divided by that sum. 7. The graph provides sorted route-time estimates. 8. The route builds JSON. Browser checks request sequence, stores newest successful payload and renders dashboard fields/charts/tables.

11. API failure behavior and proposed improvements

VERIFIED IMPLEMENTATION

• Date/time ValueError becomes 400 JSON.

• Service closure does not reject the request; it returns a closed state and zero selected demand.

• Graph errors are uncaught in JSON API, unlike /get_footfall.

• Endpoint does not check line-filter membership.

POTENTIAL BUG OR RISK

• Wrong JSON types can raise AttributeError/TypeError at .strip() or later; body fields are not schema-validated.

• Unknown station can reach empty-line optimizer and fail at max(line_recommendations, ...).

• Unknown line silently leaves primary recommendation selected.

• API response mixes null, numeric values and strings in dependent fields; clients must honor service status.

• Recomputing graph shortest paths per request may become costly if topology expands; currently small graph and no cache.

PROPOSED IMPROVEMENT

Validate JSON/query input into a schema before business logic; validate station and line against loaded metadata; use one error response shape; log server errors; decide whether travel-time output should remain available when closed; add boundary tests around service-hour edges and aggregate thresholds. Do not describe these improvements as already implemented.

12. Interview questions with follow-ups

Why calculate hourly and minute demand separately? The model is trained on hourly examples. Minute interpolation is a presentation estimate between neighboring hourly outputs; schedule decisions currently use the selected whole-hour count.

Why is the selected-line KPI not the entire station? The chosen line's recommendation replaces the primary headline; for line-specific display the frontend scales selected-time estimate using the line/hour demand share. All-lines interchange view aggregates capacity/frequency separately.

Why set closed utilization to null instead of zero? Zero utilization could incorrectly imply available service capacity with no passengers. null indicates the ratio is not meaningful when capacity is zero/not operating.

What does "no collision" mean in this API? Nothing: the app has no train positions or movement simulation. Scheduling is a headway/capacity scenario, not collision avoidance.

How would you debug a 500 for an unknown station? Reproduce API input, follow get_station_info fallback to empty lines, inspect optimize_station_schedule loop and subsequent max(line_recommendations, ...), then add request-boundary station validation and a regression test.

13. Modification exercise and solution

Exercise: Ensure an unknown station returns a structured 400 before schedule evaluation.

Solution outline: After reading and normalizing station, check station in stations_df['Station'].values or a precomputed set. If false, return jsonify({"status":"error","message":"Unknown station."}), 400. Prefer a precomputed set at module initialization to avoid scanning the column each request. Add tests for missing station default, valid station, unknown string, and non-string station input. The repository currently contains no automated test suite, so these would be proposed tests, not existing tests.

Source connections

Stage	Exact location	Produces/consumes
Browser request	app/static/js/dashboard.js:154-201	JSON request values and returned response
Input/route orchestration	app/routes.py:234-398	Full response contract
Calendar/service gate	app/calendar_context.py:85-111; app/operating_hours.py:7-10	Date context and open/closed flag
Feature prediction/profile	app/forecasting.py:56-189	selected estimates and 19-point profile
Line schedule	app/scheduling_optimizer.py:252-306 and per-line helper	headline and line schedule records
Graph travel estimates	app/travel_graph.py:139-167	destination times
Browser rendering	app/static/js/dashboard.js:203-544	KPIs, charts, line rows and travel table

Appendix A - complete source excerpts

These excerpts come from the documented commit. Original line numbers are printed at left. For print readability, Unicode dash variants are normalized to ASCII hyphens and other non-ASCII source symbols are shown as explicit Unicode escape notation; this does not change the source line numbers or code meaning. Long lines wrap with indented continuations.

Complete main JSON endpoint `app/routes.py:234-398`

```
234 @app.route('/api/predict', methods=['GET', 'POST'])
235 def api_predict():
236     """
237     JSON API for dynamic frontend interaction without full page reloads.
238     """
239     data = request.get_json(silent=True) or request.args
240     station = data.get('station', 'Rajiv Chowk').strip().rstrip('†‡*#¤').strip()
241     date_str = data.get('date', '')
242     time_str = data.get('time', '09:00')
243     line_filter = data.get('line', '')
244 
245     try:
246         dt_obj, hour, minute = _parse_datetime(date_str, time_str)
247     except ValueError as exc:
248         return jsonify({"status": "error", "message": str(exc)}), 400
249     calendar_context = get_calendar_context(dt_obj.date())
250     service_open = is_service_operating(hour, minute)
251     calendar_context['service_open'] = service_open
252     st_info = get_station_info(station)
253 
254     # 1. Predictions
255     time_footfall = predict_footfall_with_minute_interpolation(station, dt_obj, hour, minute) if service_open
    else 0
256     hourly_footfall = predict_hourly_footfall(station, dt_obj, hour) if service_open else 0
257     profile_24h = predict_station_full_day_profile(station, dt_obj)
258 
259     # 2. Scheduling Optimization
260     schedule_res = optimize_station_schedule(
261         station=station,
262         primary_line=st_info['line'],
263         all_lines=st_info['all_lines'],
264         hour=hour,
265         day_of_week=dt_obj.weekday(),
266         total_footfall=hourly_footfall,
267         is_workday=calendar_context['is_workday'],
268         service_open=service_open
269     )
270 
271     # Filter if requested
272     filtered_rec = schedule_res['primary_recommendation']
273     station_summary = not line_filter and schedule_res['is_interchange']
274     if line_filter:
275         for r in schedule_res['line_recommendations']:
276             if r['line'].lower() == line_filter.lower():
277                 filtered_rec = r
278                 break
279     elif station_summary:
280         # The demand chart/KPI describe the whole interchange. Aggregate service
281         # across every corridor too, so the headline never compares station-wide
282         # passengers with just one line's trains or capacity.
283         recommendations = schedule_res['line_recommendations']
284         total_demand = schedule_res['total_station_footfall']
285         recommended_capacity = sum(r['recommended_clearance_capacity'] for r in recommendations)
286         existing_capacity = sum(r['existing_clearance_capacity'] for r in recommendations)
287         utilization = round(total_demand / recommended_capacity * 100, 1) if recommended_capacity else 0.0
288         if utilization > 105.0:
289             risk, badge, color = 'CRITICAL_OVERCROWDING', 'Severe Crowd Alert', '#dc3545'
290         elif utilization > 88.0:
291             risk, badge, color = 'HIGH_CROWDING', 'High Demand', '#fd7e14'
292         elif utilization > 65.0:
293             risk, badge, color = 'MODERATE', 'Optimal Load', '#28a745'
294         else:
295             risk, badge, color = 'LOW', 'Comfortable Capacity', '#17a2b8'
296         total_trains = lambda key: round(sum(r[key] for r in recommendations), 1)
297         total_fleet = sum(r['fleet_size_available'] for r in recommendations)
298         demand_weight = max(total_demand, 1)
299         filtered_rec = {
300             **schedule_res['primary_recommendation'],
301             'line': 'All lines',
302             'line_demand': total_demand,
303             'is_station_summary': True,
304             'line_count': len(recommendations),
305             'recommended_headway_min': None,
306             'existing_headway_min': None,
307             'recommended_trains_per_hour': total_trains('recommended_trains_per_hour'),
308             'existing_trains_per_hour': total_trains('existing_trains_per_hour'),
309             'recommended_clearance_capacity': recommended_capacity,
310             'existing_clearance_capacity': existing_capacity,
311             'capacity_utilization_pct': utilization,
312             'crowding_risk': risk,
313             'crowding_badge': badge,
314             'crowding_color': color,
315             'avg_wait_time_rec_min': round(sum(r['avg_wait_time_rec_min'] * r['line_demand'] for r in
    recommendations) / demand_weight, 1),
316             'trains_deployed_rec': sum(r['trains_deployed_rec'] for r in recommendations),
317             'fleet_size_available': total_fleet,
318             'explanation': (
319                 f"Combined estimate across {len(recommendations)} lines: {total_demand:,} passengers/hr. "
320                 f"The recommended schedules provide an estimated comfortable boarding capacity of "
321                 f"{recommended_capacity:,} passengers/hr ({utilization}% utilization). "
322                 "This is a model estimate based on configurable train-capacity assumptions, not a live passenger
     count."
323             )
324         }
325 
326     if not service_open:
```

```
        filtered_rec = {
            **filtered_rec,
            'line': 'All lines' if station_summary else filtered_rec['line'],
            'line_demand': 0,
            'recommended_headway_min': None,
            'existing_headway_min': None,
            'recommended_trains_per_hour': 0,
            'existing_trains_per_hour': 0,
            'recommended_clearance_capacity': 0,
            'existing_clearance_capacity': 0,
            'capacity_utilization_pct': None,
            'crowding_risk': 'SERVICE_CLOSED',
            'crowding_badge': 'Metro closed',
            'crowding_color': '#64748b',
            'avg_wait_time_rec_min': None,
            'trains_deployed_rec': 0,
            'status_action': 'SERVICE_CLOSED',
            'explanation': f"Metro service is closed at {hour:02d}:{minute:02d} (modeled operating window:
05:00-23:30). No trains are operating and modeled passenger demand is zero."
        }

    # 3. Dijkstra Travel Times to ALL stations
    times = shortest_times_from(travel_graph, station)
    station_line_map = dict(zip(stations_df['Station'], stations_df['Line']))
    travel_list = [
        {
            'destination': dest,
            'travel_time_min': round(mins, 1),
            'line': station_line_map.get(dest, 'Metro Network')
        }
        for dest, mins in sorted(times.items(), key=lambda kv: kv[1])
    ]

    calendar_context['hour_band'] = _hour_band(profile_24h['predicted'], hour, minute)
    calendar_context['selected_hour_demand'] = hourly_footfall
    calendar_context['daily_peak_hour'] = profile_24h['peak_hour']

    return jsonify({
        "status": "success",
        "station": station,
        "station_info": st_info,
        "datetime": {
            "date": dt_obj.strftime('%Y-%m-%d'),
            "day": dt_obj.strftime('%A'),
            "time": f"{hour:02d}:{minute:02d}",
            "hour": hour,
            "minute": minute,
            "is_peak": filtered_rec['is_peak'],
            "service_open": service_open
        },
        "calendar_context": calendar_context,
        "footfall": {
            "instant_interpolated": time_footfall,
            "hourly_demand": hourly_footfall,
            "daily_total": profile_24h['daily_total'],
            "station_peak_hour": profile_24h['peak_hour'],
            "station_peak_footfall": profile_24h['peak_footfall'],
        },
        "scheduling": {
            "headline": filtered_rec,
            "all_lines": schedule_res['line_recommendations'],
            "is_interchange": schedule_res['is_interchange']
        },
        "profile_24h": profile_24h,
        "travel_times": travel_list,
        "data_provenance": {
            "station_topology": "PROJECT_ROUTE_MAP_DATA_REQUIRES_SOURCE_VERIFICATION",
            "train_capacities": "CONFIGURABLE_PROJECT_ASSUMPTIONS_NOT_VERIFIED_DMRC_SPECIFICATIONS",
            "baseline_headways": "CONFIGURABLE_PROJECT_ASSUMPTIONS_NOT_VERIFIED_DMRC_TIMETABLES",
            "hourly_footfall_forecast": "MODEL_ESTIMATE_TRAINED_ON_SYNTHETIC_SIMULATION_NOT_OBSERVED_COUNTS",
            "interchange_demand_distribution": "SYNTHETIC_WEIGHTED_ESTIMATE_NOT_OBSERVED_LINE_SPLIT"
        }
    })
```

Station directory JSON endpoint `app/routes.py:401-415`

```
@app.route('/api/stations')
def api_stations():
    stations_data = []
    for _, row in stations_df.iterrows():
        st = row['Station']
        all_lines = [l.strip() for l in str(row.get('All_Lines', row['Line'])).split(',')]
        stations_data.append({
            "station": st,
            "line": row['Line'],
            "all_lines": all_lines,
            "is_interchange": bool(row.get('Is_Interchange', len(all_lines) > 1)),
            "latitude": float(row['Latitude']) if pd.notna(row['Latitude']) else None,
            "longitude": float(row['Longitude']) if pd.notna(row['Longitude']) else None,
        })
    return jsonify({"stations": stations_data})
```

Metrics JSON endpoint `app/routes.py:418-420`

```
@app.route('/api/model_metrics')
def api_model_metrics():
    return jsonify(get_model_evaluation_summary())
```

Complete browser fetchPrediction() function `app/static/js/dashboard.js:154-201`

```
154 async function fetchPrediction() {
155     const requestId = ++predictionRequestId;
156     const station = document.getElementById('station').value;
157     const date = document.getElementById('date').value;
158     const time = document.getElementById('time').value;
159     const line = document.getElementById('lineSelect').value;
160 
161     const loader = document.getElementById('loadingIndicator');
162     const formStatus = document.getElementById('forecastFormStatus');
163     if (loader) loader.style.display = 'inline-block';
164     if (formStatus) {
165         formStatus.textContent = 'Updating analysis…';
166         formStatus.classList.remove('is-error');
167     }
168 
169     try {
170         const res = await fetch('/api/predict', {
171             method: 'POST',
172             headers: { 'Content-Type': 'application/json' },
173             body: JSON.stringify({ station, date, time, line })
174         });
175         const data = await res.json();
176         if (requestId !== predictionRequestId) return;
177         if (data.status === 'success') {
178             currentData = data;
179             renderDashboard(data);
180             document.body.classList.add('forecast-ready');
181             if (formStatus) formStatus.textContent = '';
182             requestAnimationFrame(() => {
183                 if (demandChart) demandChart.resize();
184                 if (lineChart) lineChart.resize();
185             });
186         } else if (formStatus) {
187             formStatus.textContent = data.message || 'The forecast could not be generated. Check the selected
    inputs and try again.';
188             formStatus.classList.add('is-error');
189         }
190     } catch (err) {
191         console.error('Error fetching prediction:', err);
192         if (formStatus) {
193             formStatus.textContent = 'We could not reach the forecasting service. Please try again.';
194             formStatus.classList.add('is-error');
195         }
196     } finally {
197         if (requestId === predictionRequestId) {
198             if (loader) loader.style.display = 'none';
199         }
200     }
201 }
```

Station-level optimizer output contract `app/scheduling_optimizer.py:252-306`

```
252 def optimize_station_schedule(station: str, primary_line: str, all_lines: list,
253                              hour: int, day_of_week: int, total_footfall: int,
254                              is_workday=None, service_open=True) -> dict:
255     """
256     Evaluates demand and scheduling for the entire station, including line-wise breakdowns
257     for interchange stations.
258     """
259     is_interchange = len(all_lines) > 1
260     line_splits = get_interchange_line_split(station, all_lines) if is_interchange else {primary_line: 1.0}
261 
262     line_recommendations = []
263     for line in all_lines:
264         share = line_splits.get(line, 1.0 / len(all_lines))
265         line_demand = round(total_footfall * share)
266         line_rec = optimize_line_schedule(
267             line, station, hour, day_of_week, line_demand if service_open else 0, is_workday=is_workday
268         )
269         if not service_open:
270             line_rec.update({
271                 'line_demand': 0,
272                 'existing_headway_min': None,
273                 'existing_trains_per_hour': 0,
274                 'existing_clearance_capacity': 0,
275                 'existing_line_capacity': 0,
276                 'recommended_headway_min': None,
277                 'recommended_trains_per_hour': 0,
278                 'recommended_clearance_capacity': 0,
279                 'recommended_total_line_capacity': 0,
280                 'capacity_shortfall_surplus': 0,
281                 'capacity_utilization_pct': 0,
282                 'avg_wait_time_existing_min': 0,
283                 'avg_wait_time_rec_min': 0,
284                 'trains_deployed_rec': 0,
285                 'crowding_risk': 'SERVICE_CLOSED',
286                 'crowding_badge': 'Metro closed',
287                 'crowding_color': '#64748b',
288                 'status_action': 'SERVICE_CLOSED',
289                 'explanation': 'Metro service is closed at the selected time. No trains are operating and
    modeled passenger demand is zero.'
290             })
291         line_rec["demand_share_pct"] = round(share * 100.0, 1)
292         line_recommendations.append(line_rec)
293 
294     # Aggregate summary (using primary line or highest-demand line)
295     primary_rec = max(line_recommendations, key=lambda r: r['line_demand'])
296 
297     return {
298         "station": station,
299         "is_interchange": is_interchange,
300         "total_station_footfall": total_footfall if service_open else 0,
301         "all_lines": all_lines,
302         "primary_line": primary_rec["line"],
303         "line_distribution_metadata": _LINE_DIST_MODEL.get("metadata", {}),
304         "line_recommendations": line_recommendations,
305         "primary_recommendation": primary_rec
306     }
```